



Deep Learning

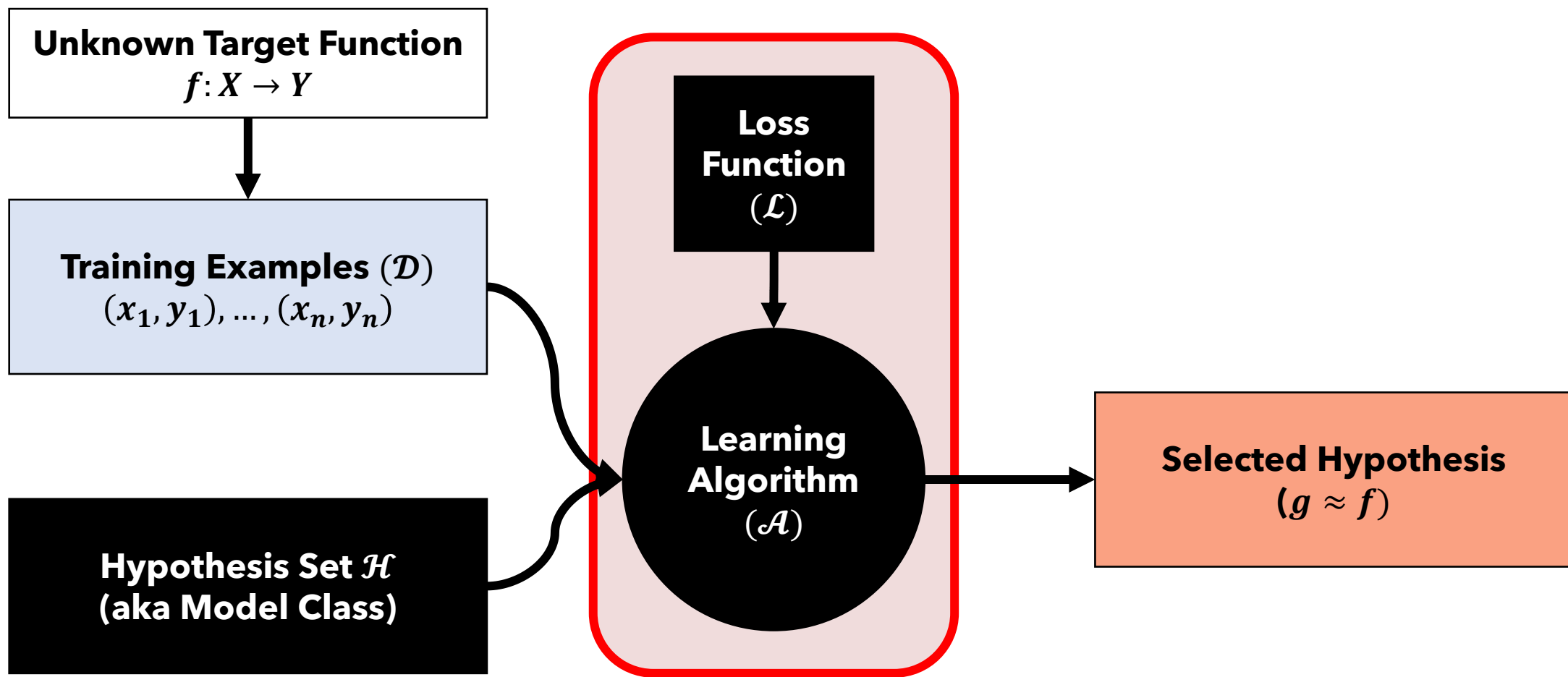
Lecture 3.2: Neural Network Optimization (I)





RECAP

From Last Lecture



Observations of some phenomenon



Designed by us



Result of our learning



Loss Function - A function measuring “how bad” a model’s output is, usually relative to some gold-standard for what the output should be.

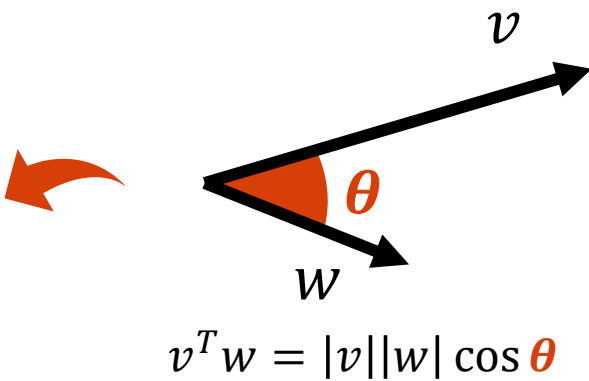
Typically, we want to find model parameters θ that minimize the loss over all our datapoints:

$$\min_{\theta} \sum_{i=0}^n \text{loss}(y_i, f(x_i; \theta))$$

1st Order Taylor Series Approximation: $f(x) \approx f(a) + \nabla f(a)^T (x - a)$

Consider moving from a to a new parameter value $a + \Delta x$, what choice of Δx minimizes $f(x)$?

Our Taylor approximation suggests: $f(a + \Delta x) \approx f(a) + \nabla f(a)^T \Delta x$

$$\min_{\Delta x} f(a) + \underbrace{\nabla f(a)^T \Delta x}_{\Delta x = -\alpha \nabla f(a)}$$


$$v^T w = |v||w| \cos \theta$$

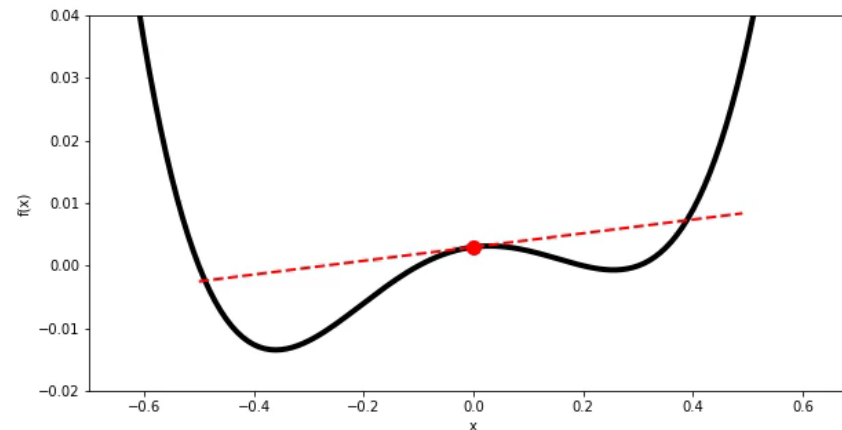
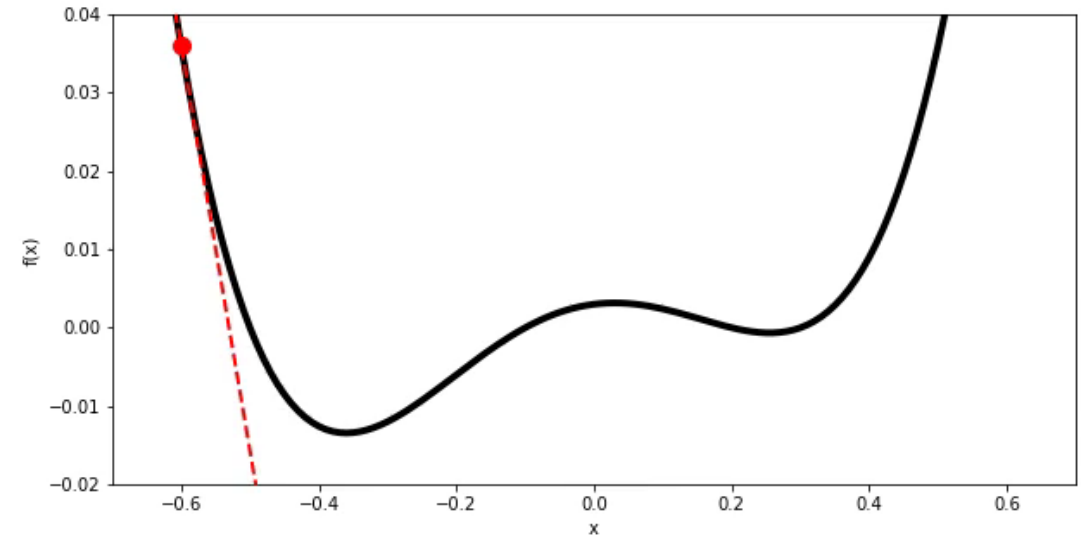
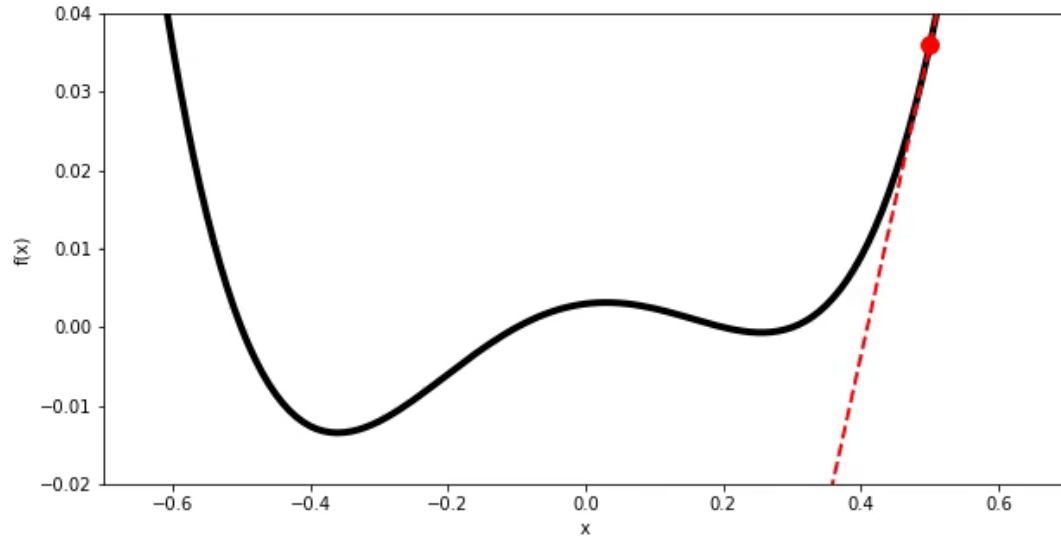
What value for α minimizes $f(a - \alpha \nabla f(a))$?

But wait! Our Taylor series approximation is only valid near a ! Deciding on the step-size α is tricky business.

Gradient descent update rule:

$$x^{(t+1)} = x^{(t)} - \alpha \nabla f(x^{(t)})$$

$$\alpha > 0$$



Today's Learning Objectives

Be able to answer:

- What optimizers are used for neural networks?
 - What is stochastic gradient descent?
 - What is momentum?
 - What is weight decay?
 - What are learning rate schedules?
- How is data processed for neural networks?
 - Why should data be normalized?
 - What is data augmentation and why do it?
- What are best practices for network initialization?





We want to find model parameters θ that minimize the loss over our datapoints:

$$\min_{\theta} L(\theta) = \min_{\theta} \sum_{i=0}^n l(y_i, f(x_i; \theta))$$

Naïve application of gradient descent would suggest repeatedly applying:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla L(\theta^{(t)})$$

Why might this be inefficient for deep learning?



Overall loss is a sum of loss for each point. Gradient distributes.

$$\nabla_{\theta} L(\theta) = \sum_{i=0}^n \nabla_{\theta} l(y_i, f(x_i; \theta))$$

Each step of optimization requires processing all datapoints....

Let's approximate this.



Gradient direction estimated from our entire dataset:

$$\nabla_{\theta} L(\theta) = \sum_{i=0}^n \nabla_{\theta} l(y_i, f(x_i; \theta))$$

What if at each step, we sample a subset of our data - $B \subset \{0, 1, \dots, n\}$

$$\nabla_{\theta} L(\theta) \approx \sum_{i \in B} \nabla_{\theta} l(y_i, f(x_i; \theta))$$

Is this okay or did we ruin everything?



It averages out if we sample the batch of data randomly:

$$\nabla_{\theta} L(\theta) = E_B \left[\sum_{i \in B} \nabla_{\theta} l(y_i, f(x_i; \theta)) \right]$$

Stochastic Gradient Descent Algorithm

$\theta \leftarrow \text{random}(\quad)$

Sample batch B

while $|\nabla_{\theta} L| > \epsilon$ or iters remain

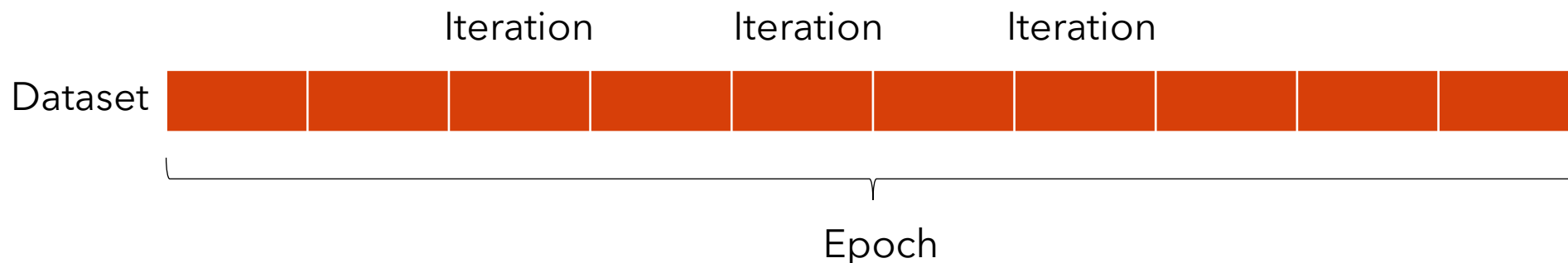
$\theta = \theta - \alpha \nabla_{\theta} L(B; \theta)$

Typically, don't *actually* sample randomly.

- Random access is inefficient from memory or from disk.

Instead, we'll repeat the following:

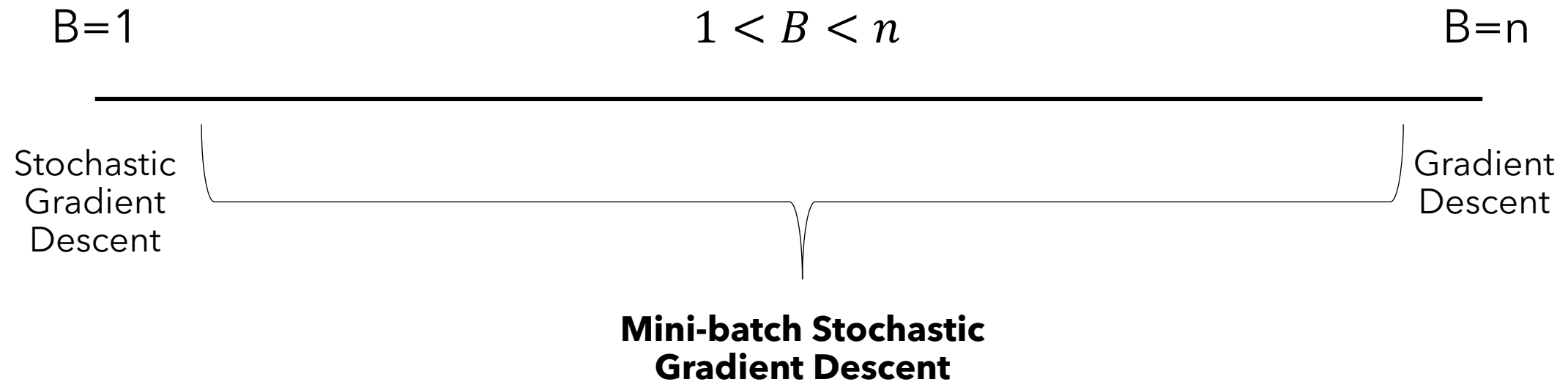
- Randomly shuffle the input examples
- Iterate through all examples one **batch** at a time (32-256 examples common, or anything you can fit in the GPU memory)
 - Define an **iteration** to be every time the gradient is computed
- After every example has been seen once, call that an **epoch**





Spectrum between Stochastic Gradient Descent and Gradient Descent

Batch size determines a spectrum. Let n be number of total examples and B be batch size.



Today's Learning Objectives

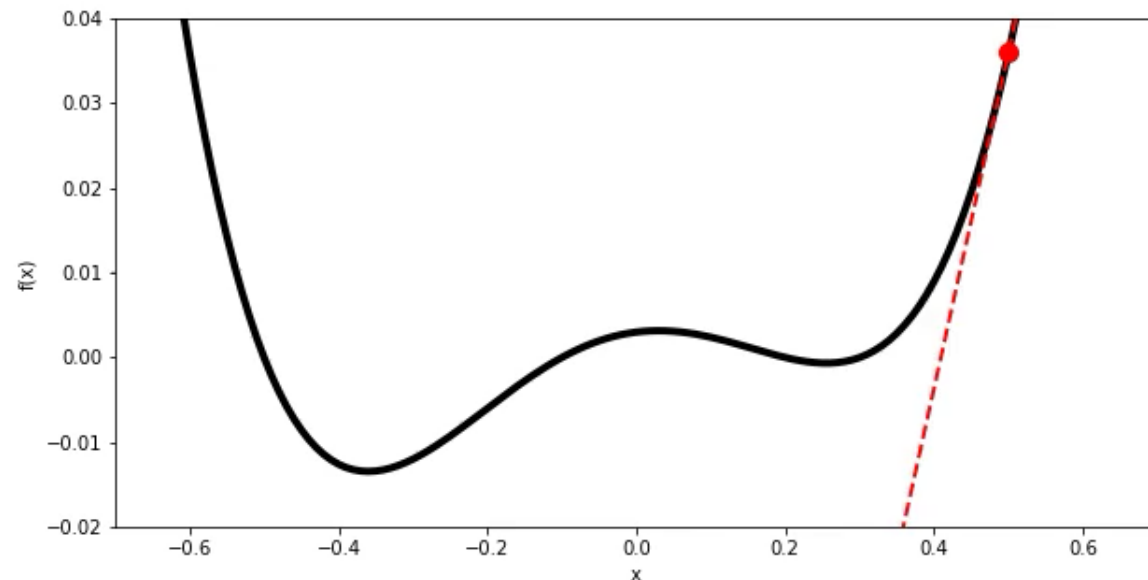
Be able to answer:

- What optimizers are used for neural networks?
 - ~~What is stochastic gradient descent?~~
 - What is momentum?
 - What is weight decay?
 - What are learning rate schedules?
- How is data processed for neural networks?
 - Why should data be normalized?
 - What is data augmentation and why do it?
- What are best practices for network initialization?





Gradient descent can stop at poor local minima or saddle points.



lr: 0.01

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla L(\theta^{(t)})$$

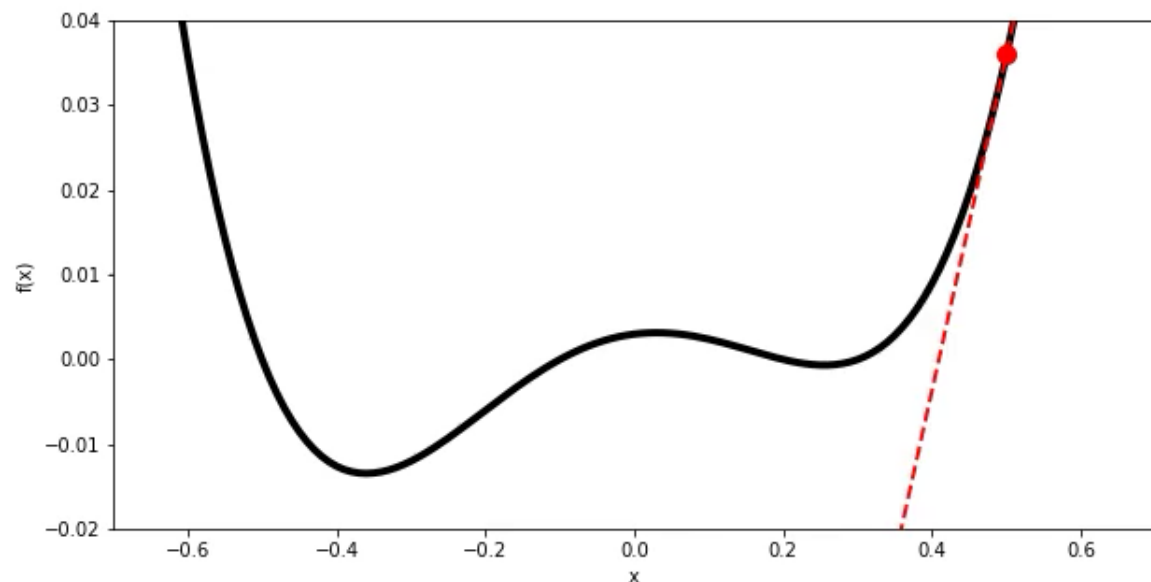
Idea: Let's give our update a bit of **momentum** so that it continues to go in directions it has been steadily going already.



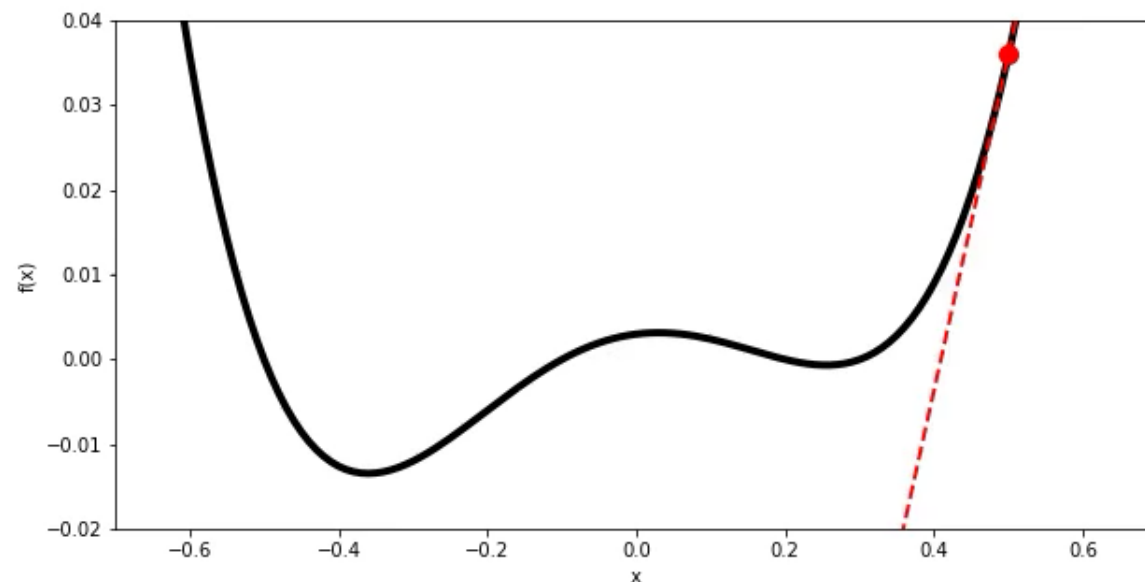
Idea: Let's give our update a bit of **momentum** so that it continues to go in directions it has been steadily going already.

$$m^{(t+1)} = \mu m^{(t)} - \alpha \nabla L(\theta^{(t)}) \quad 0 \leq \mu < 1$$

$$\theta^{(t+1)} = \theta^{(t)} + m^{(t+1)}$$



lr: 0.01



lr: 0.01, momentum=0.9



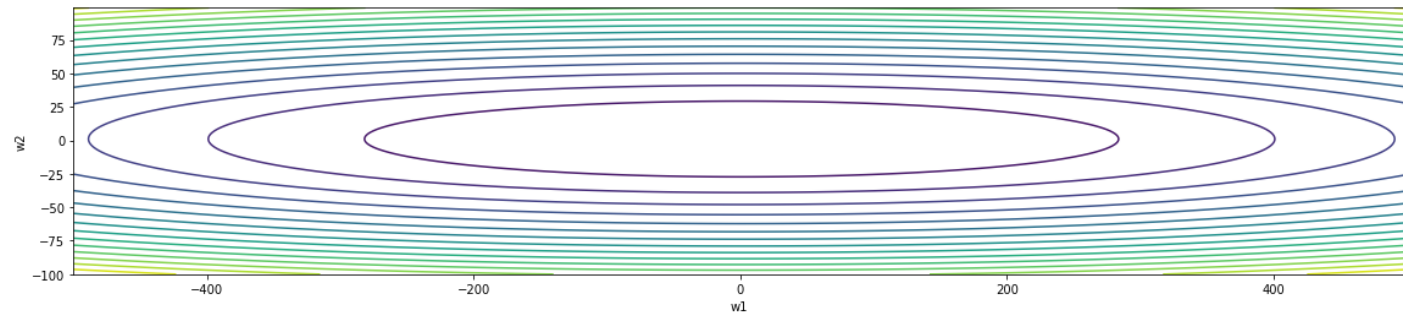
Momentum can also help reduce impact of learning rate:

- Potentially reducing oscillations at high learning rate
- Improving convergence speed of low learning rate

Let's consider our "banana boat" function again

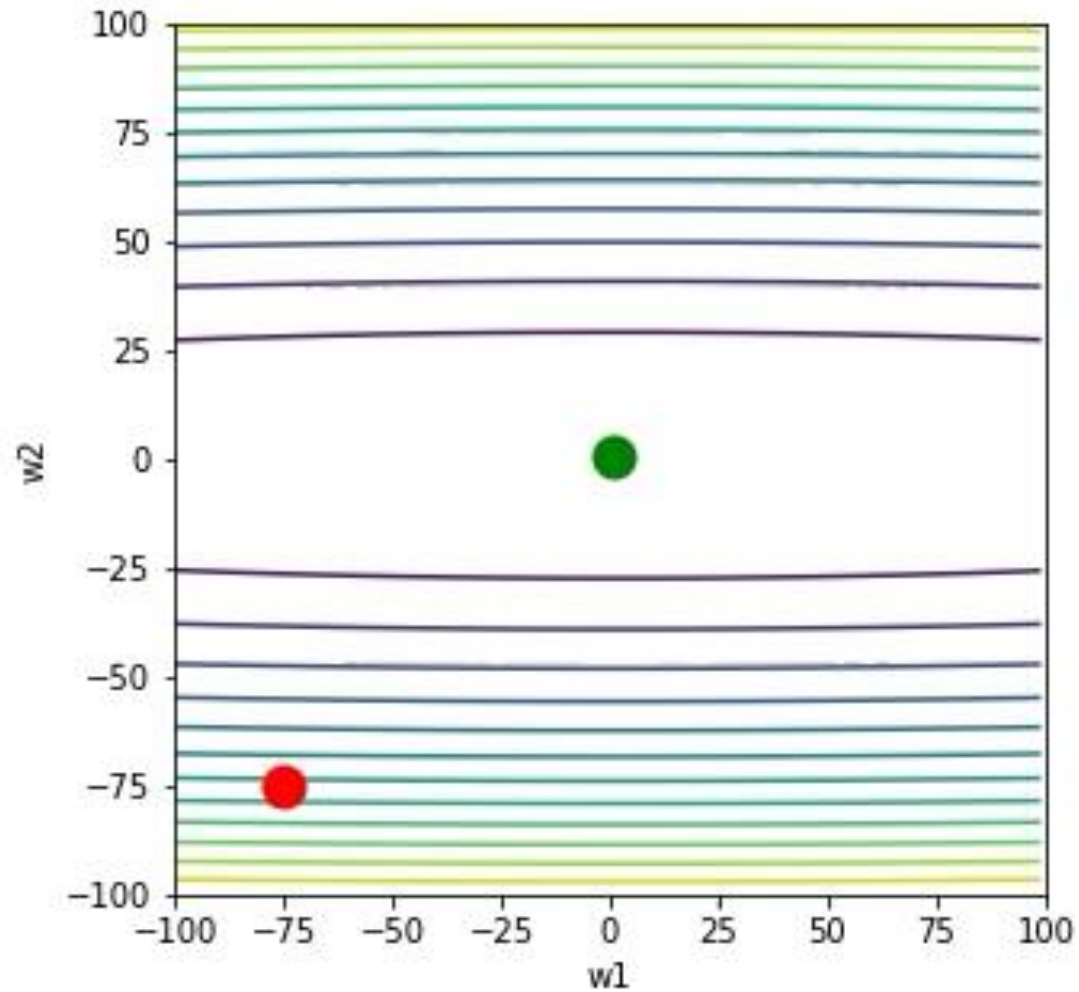


$$f(\mathbf{w}) = (w_1 - 1)^2 + 100(w_2 - 1)^2$$

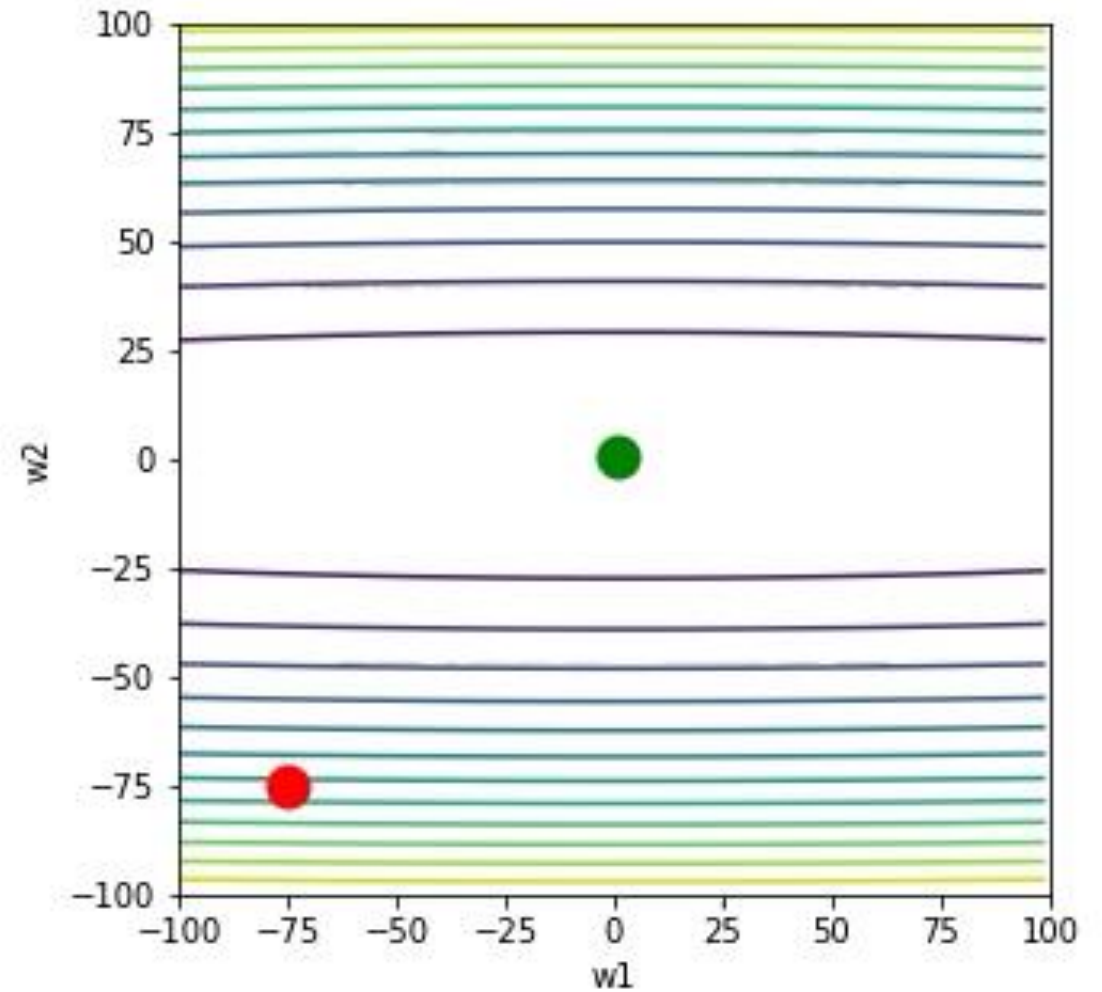




Stochastic Gradient Descent with Momentum



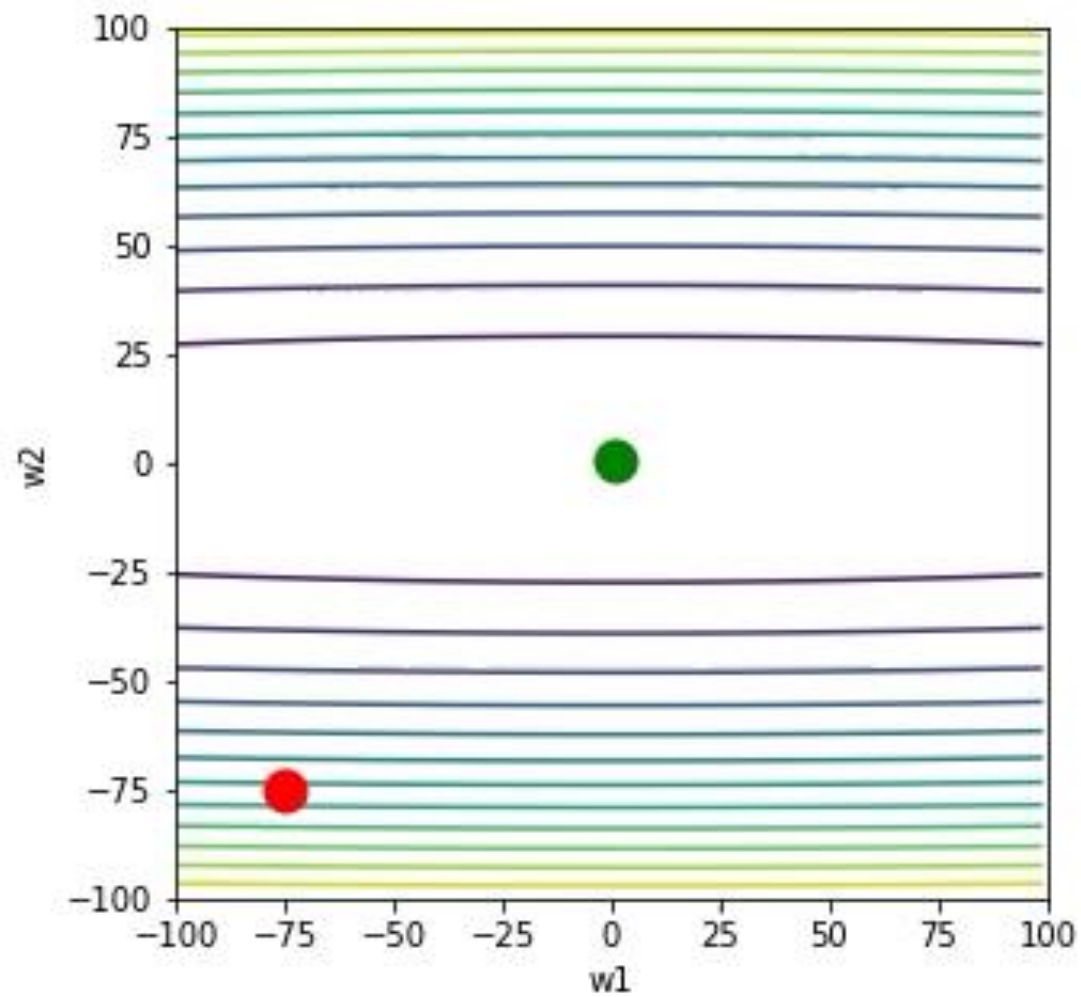
lr: 0.001



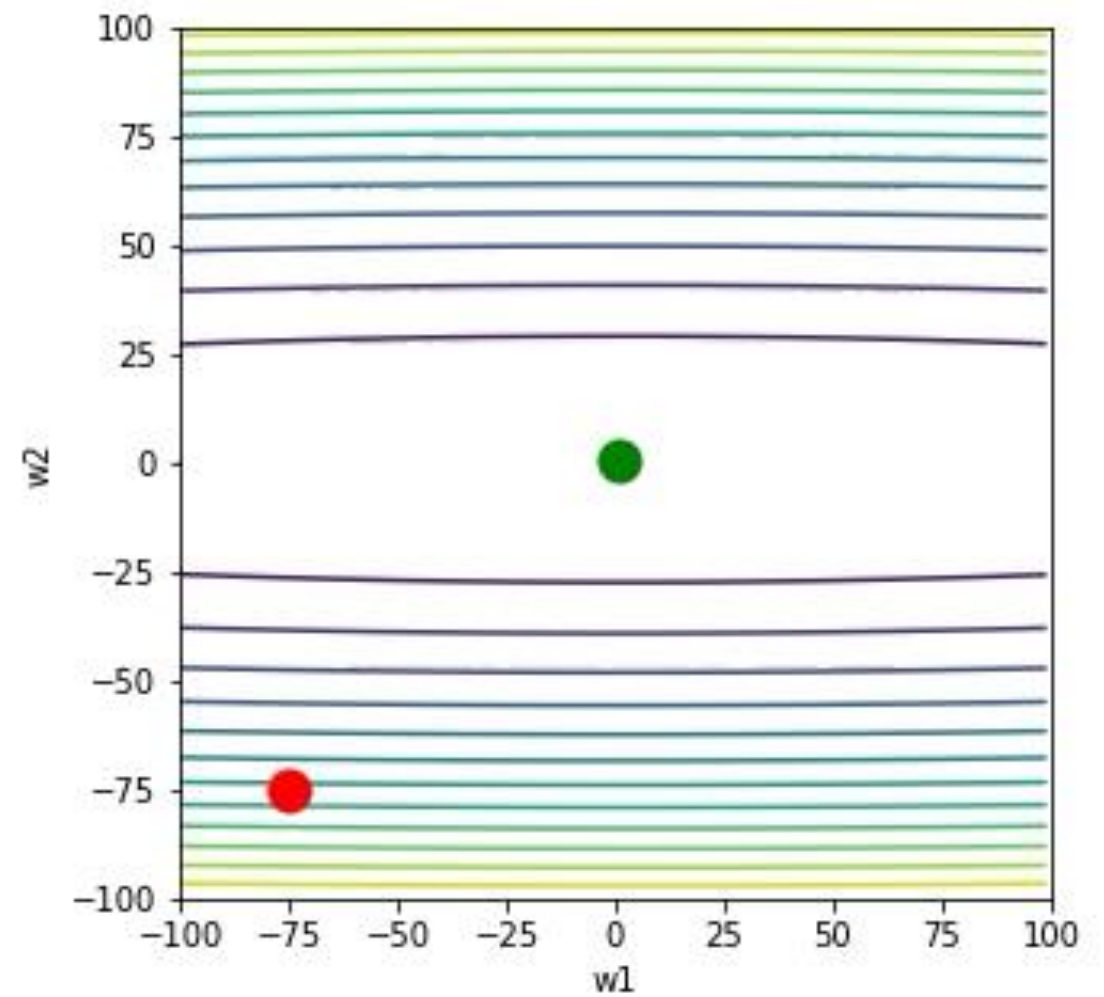
lr: 0.001,
momentum=0.9



Stochastic Gradient Descent with Momentum



$\text{lr: } 0.01$



$\text{lr: } 0.01,$
 $\text{momentum}=0.9$



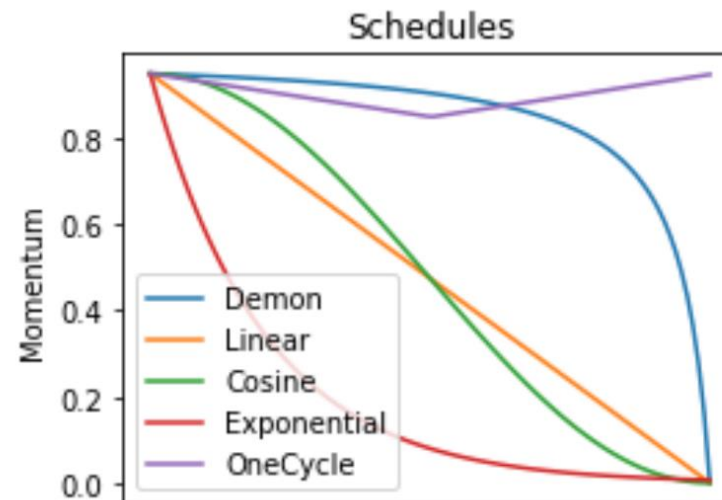
Stochastic Gradient Descent with Momentum

Typically set to 0.8 - 0.9 when momentum is used

- This is a rule-of-thumb and not justified theoretically.

Work on trying to set this right, or to change it over the course of training.

- Rise of adaptive learning rate methods have reduced this line of work.



Today's Learning Objectives

Be able to answer:

- What optimizers are used for neural networks?
 - ~~What is stochastic gradient descent?~~
 - ~~What is momentum?~~
 - What are weight decay and early stopping?
 - What are learning rate schedules?
- How is data processed for neural networks?
 - Why should data be normalized?
 - What is data augmentation and why do it?
- What are best practices for network initialization?



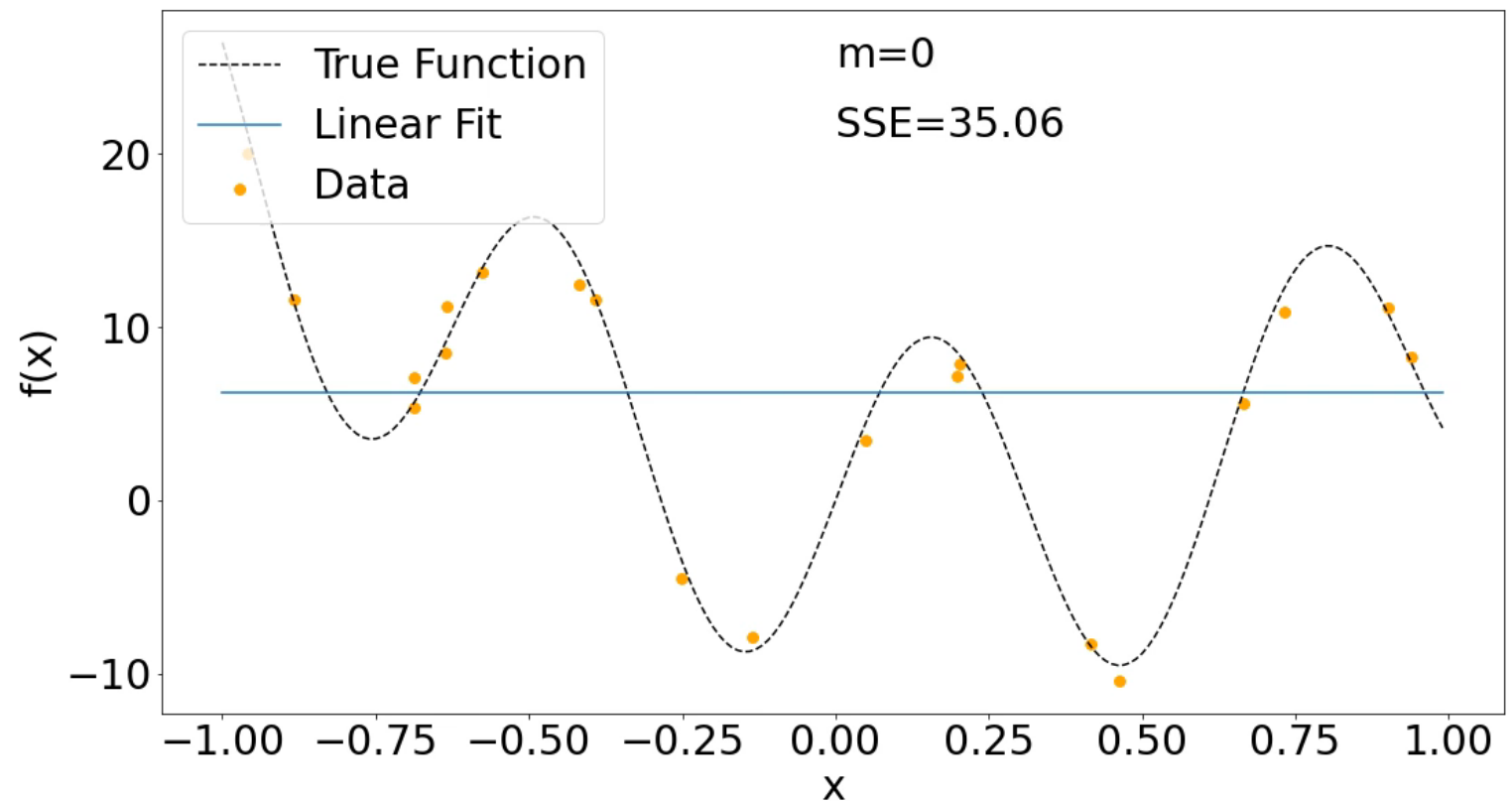


How to avoid networks from overfitting to training data?

Traditional ML would suggest adding a regularizer to encourage “simplicity”.

$$y = \sum_{i=0}^m w_i x^i$$

$$\min_w \left(y_i - \sum_{i=0}^m w_i x^i \right)^2$$

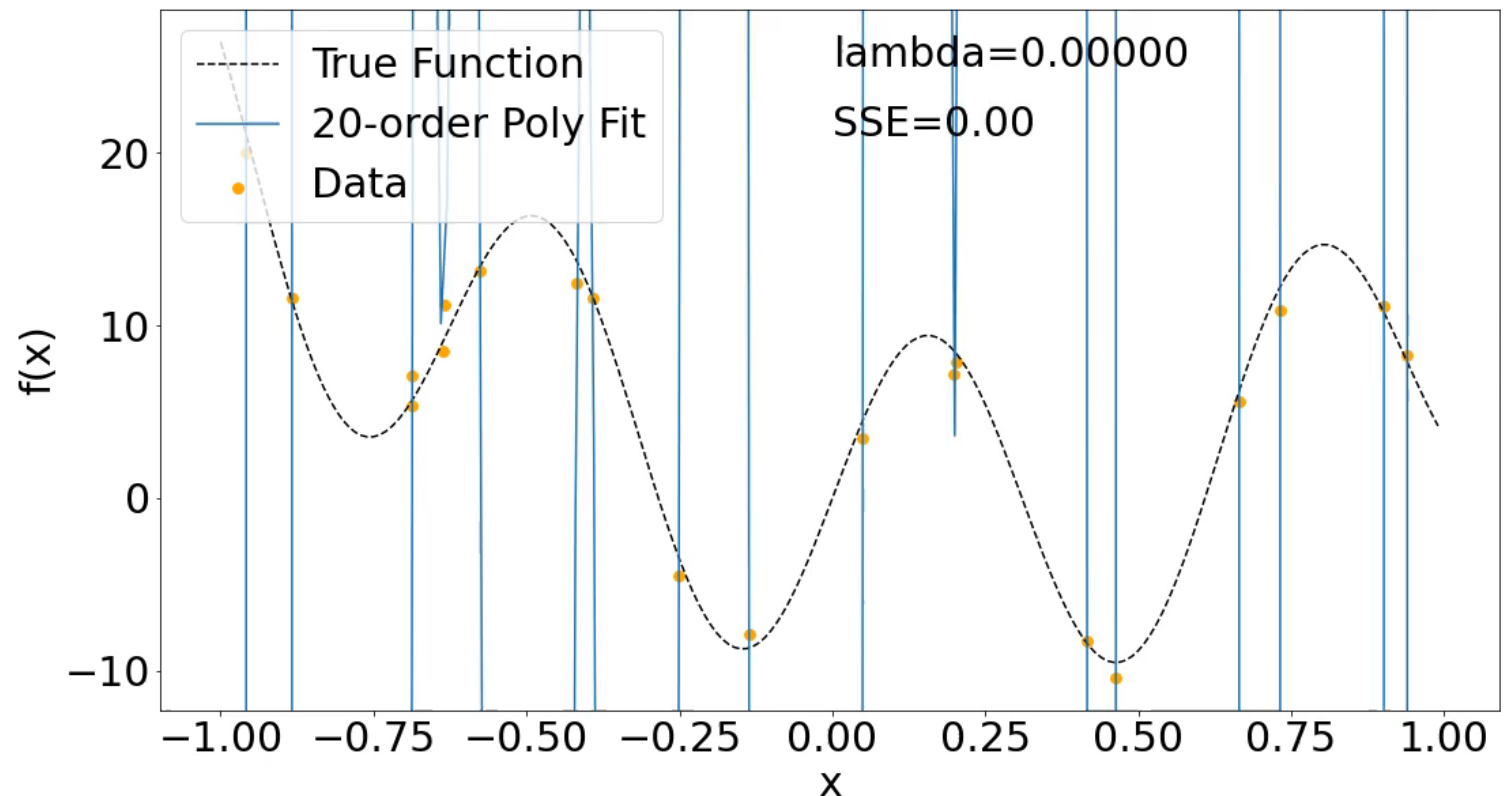




How to avoid networks from overfitting to training data?

Traditional ML would suggest adding a regularizer to encourage “simplicity”.

$$\min_w \left(y_i - \sum_{i=0}^m w_i x^i \right)^2 + \lambda w^T w$$





L2 Regularization (aka **weight decay**) for neural network:

$$\min_w \sum_{i \in B} \text{loss}(y_i, f(x_i; \theta)) + \lambda \theta^T \theta$$

Adds a term in the gradient to reduce push the weights toward zero.

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla L(\theta^{(t)}) - 2\alpha\lambda\theta$$

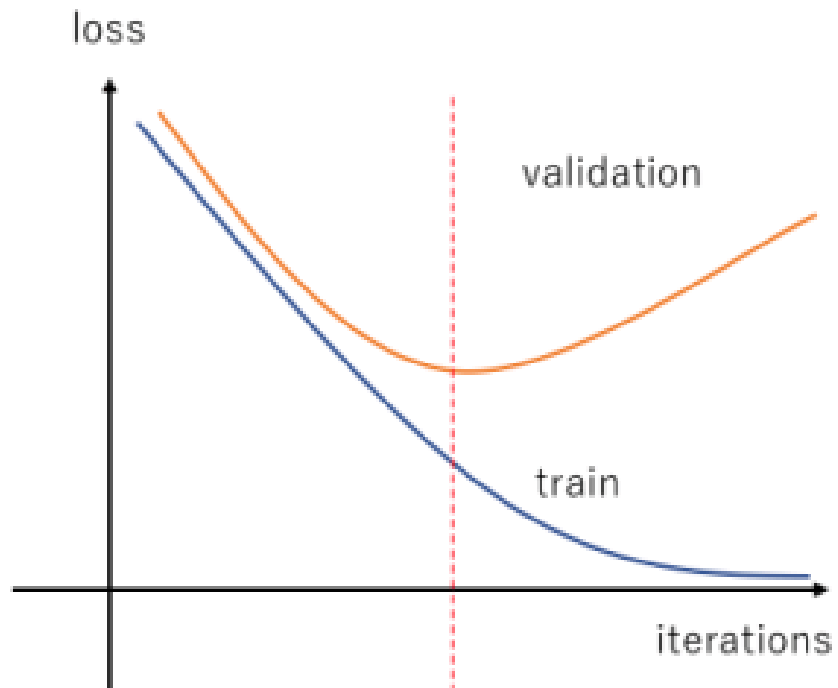
Often applied directly with just $\lambda\theta$ in the gradient step, rather than $2\alpha\lambda\theta$

**Doesn't really work all that well in deep networks until transformers....
but cheap to add.**



How to avoid networks from overfitting to training data? **Early stopping.**

Monitor the loss on the validation set and stop training when it stops decreasing.



In practice – typically run for a fixed number of epochs and save a copy of the weights after each epoch.

Select the “checkpoint” with the best validation accuracy.

Today's Learning Objectives

Be able to answer:

- What optimizers are used for neural networks?
 - ~~What is stochastic gradient descent?~~
 - ~~What is momentum?~~
 - ~~What are weight decay and early stopping?~~
 - What are learning rate schedules?
- How is data processed for neural networks?
 - Why should data be normalized?
 - What is data augmentation and why do it?
- What are best practices for network initialization?



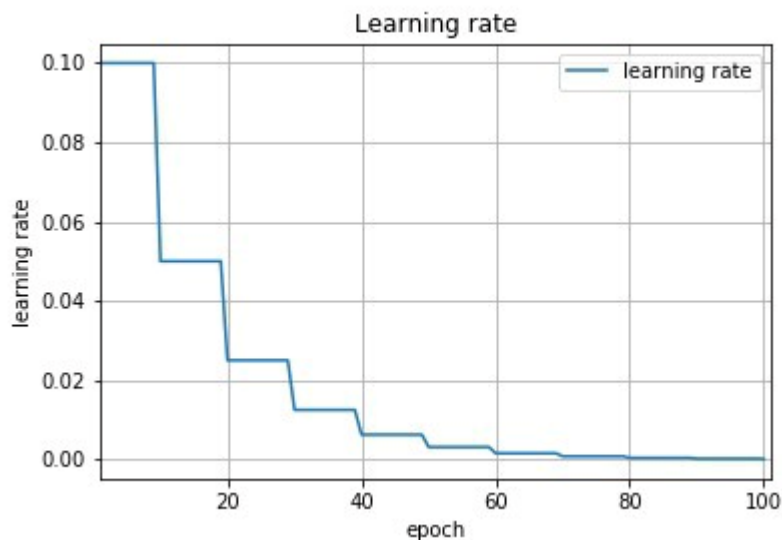


Learning Rate Schedules

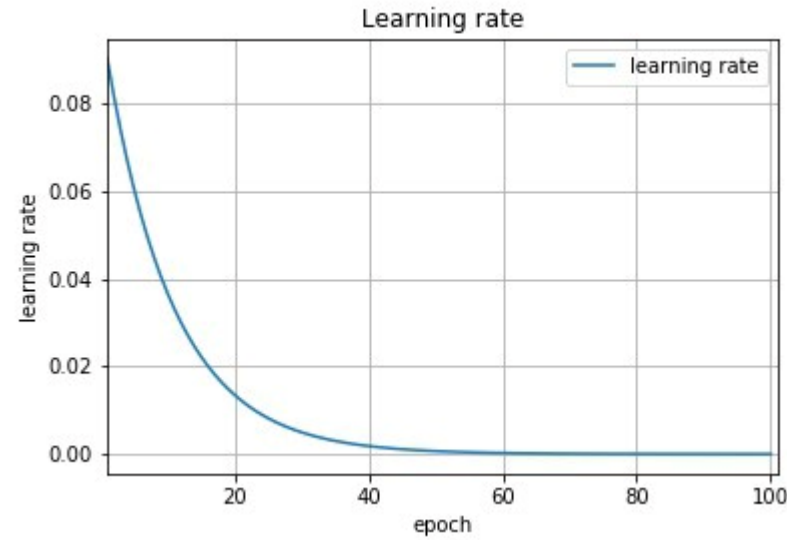
Gradient descent is only guaranteed to converge for small learning rate, but small learning rates lead to slow convergence.

Idea: Adjust learning rate over time (typically high to low, but some outliers), call how it is adjusted a **learning rate schedule**.

Typically adjust learning rate once every X per epochs



Step Decay

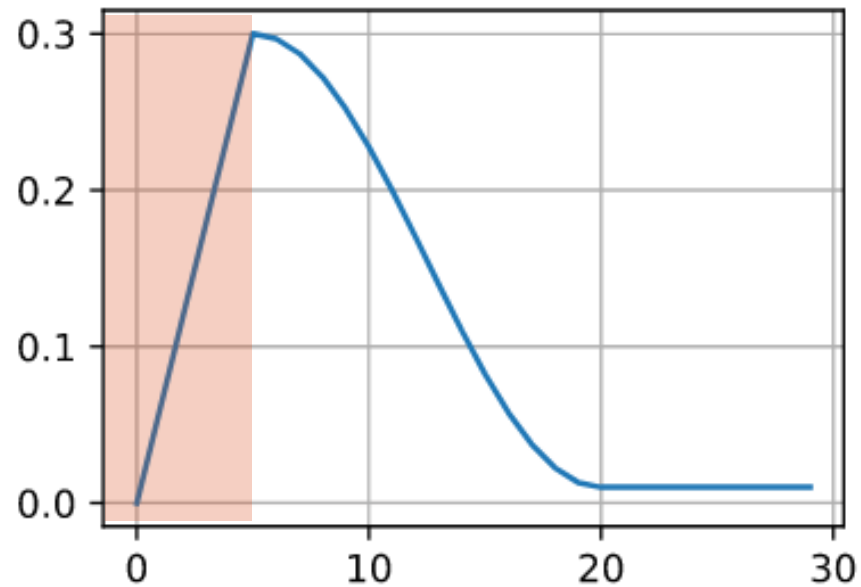


Exponential Decay



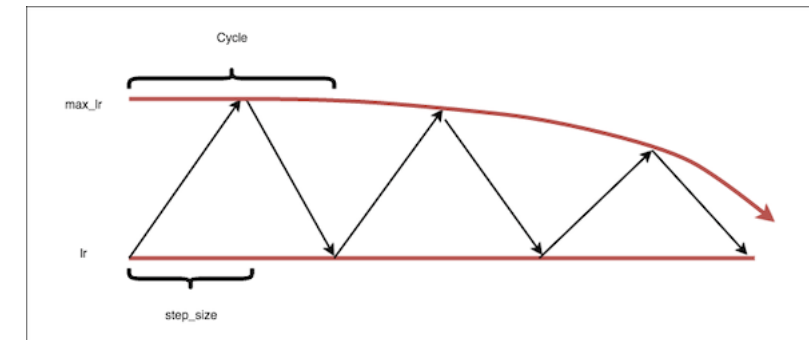
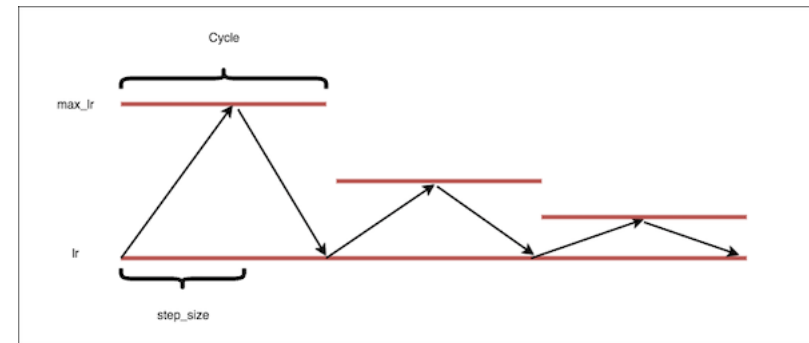
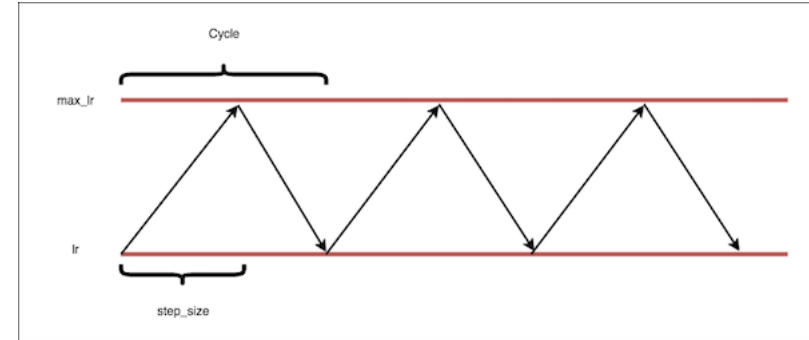
Learning Rate Schedules

Some advanced schedules:



Linear Warmup

Used when “transferring” a trained model to a new task, or training transformers



Cyclic Learning Rates

Today's Learning Objectives

Be able to answer:

- What optimizers are used for neural networks?
 - ~~What is stochastic gradient descent?~~
 - ~~What is momentum?~~
 - ~~What are weight decay and early stopping?~~
 - ~~What are learning rate schedules?~~
- How is data processed for neural networks?
 - Why should data be normalized?
 - What is data augmentation and why do it?
- What are best practices for network initialization?





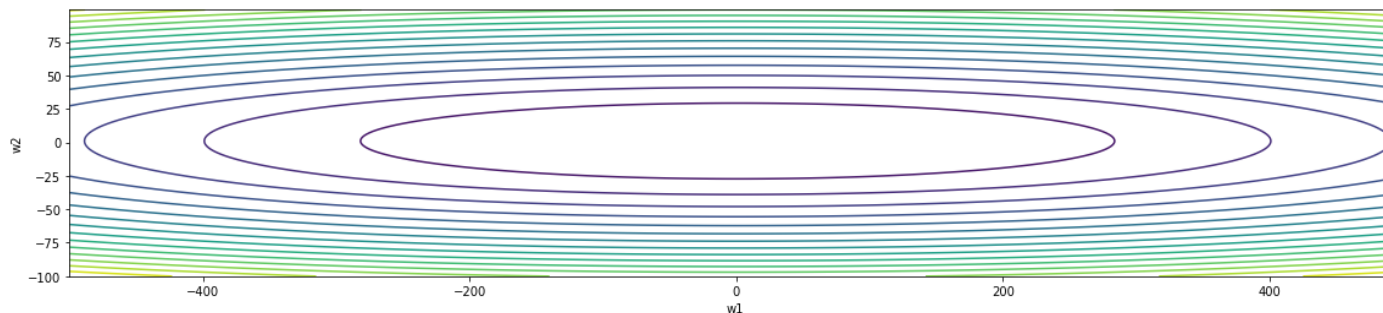
Standard practice to normalize input to neural network:

- Subtract mean and divide by standard deviation

For image data - often good enough to subtract mean and divide by max (255).

Why do this?

Want to avoid situations like this where one dimension is dominant.

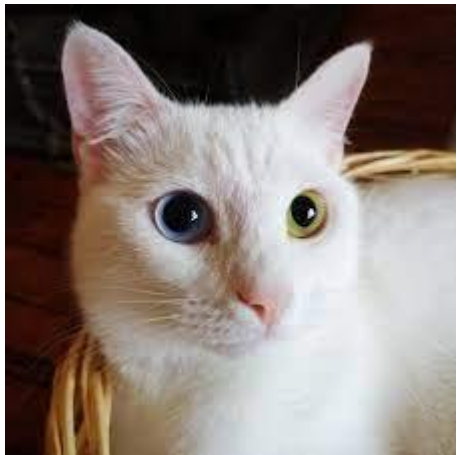


Also, keeps outputs of the first layer to a reasonable enough range.

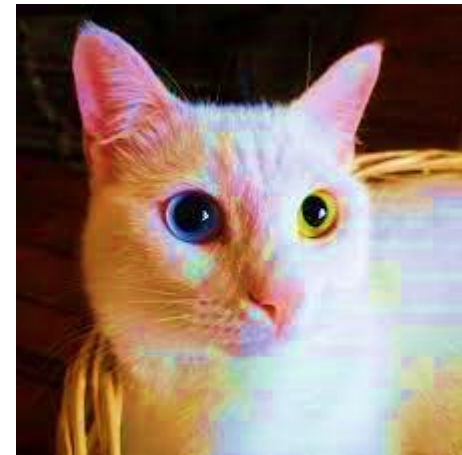
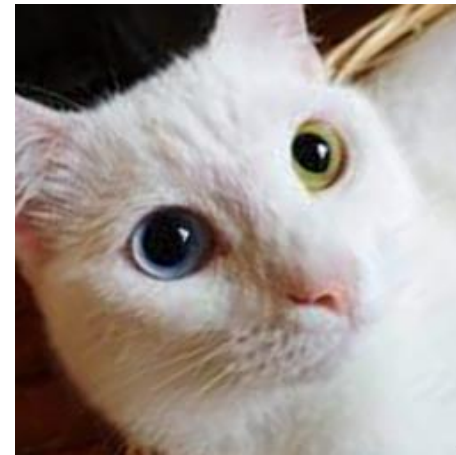


Perhaps one of the most important “tricks” in the deep learning playbook

Idea: If I can modify an input example such that it looks different but still matches its corresponding supervised output – then I’ve made a new labelled point.



Cat

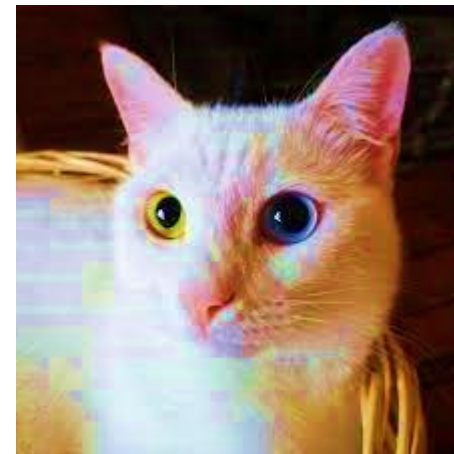
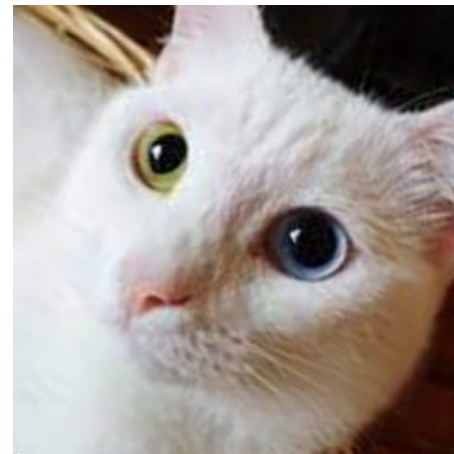
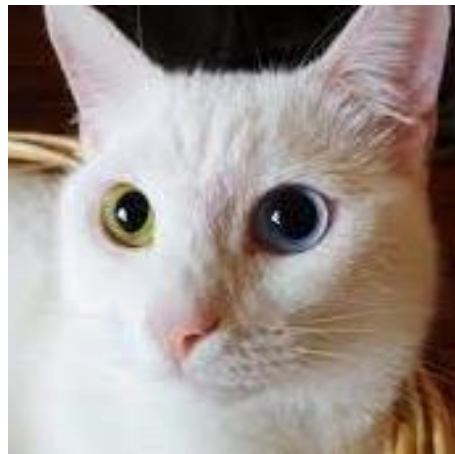
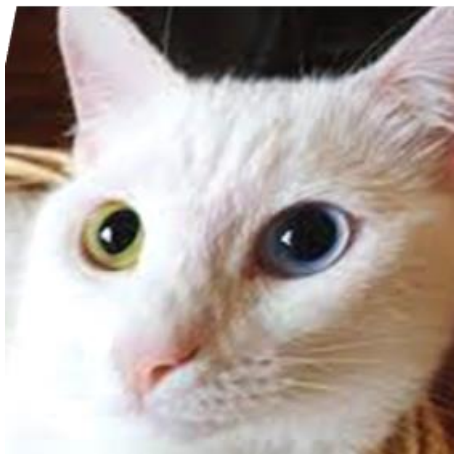
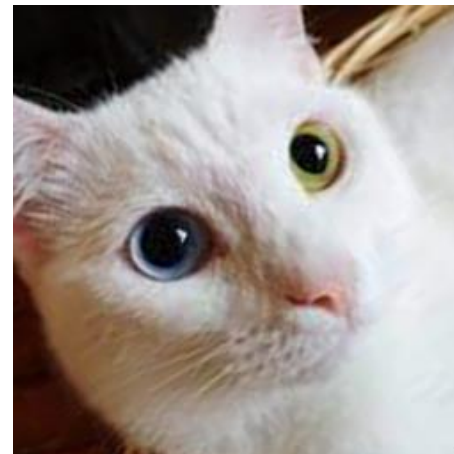
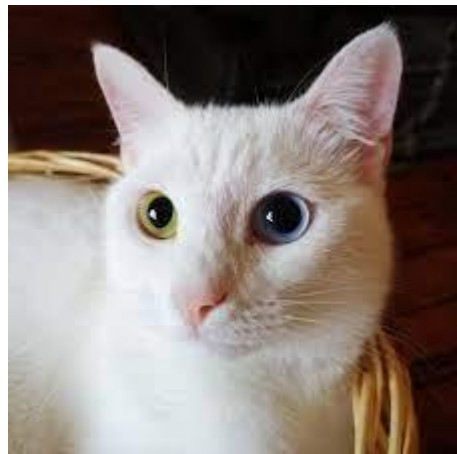




Data Augmentation



Cat

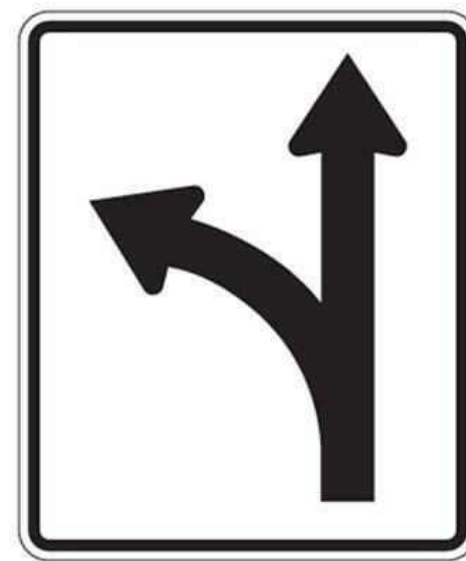




Significant variety of how to perform data augmentation in vision. Not all may preserve the match with a label. Think through these things.



Label: left-exit



Label: ?



Some work on automatically selecting which sets of augmentations to use:

arXiv:1805.09501v3 [cs.CV] 11 Apr 2019

AutoAugment: Learning Augmentation Strategies from Data

Ekin D. Cubuk*, Barret Zoph†, Dandelion Mané, Vijay Vasudevan, Quoc V. Le
Google Brain

Abstract

Data augmentation is an effective technique for improving the accuracy of modern image classifiers. However, current data augmentation implementations are manually designed. In this paper, we describe a simple procedure called **AutoAugment** to automatically search for improved data augmentation policies. In our implementation, we have designed a search space where a policy consists of many sub-policies, one of which is randomly chosen for each image in each mini-batch. A sub-policy consists of two operations, each operation being an image processing function such as translation, rotation, or shearing, and the probabilities and magnitudes with which the functions are applied. We use a search algorithm to find the best policy such that the neural network yields the highest validation accuracy on a target dataset. Our method achieves state-of-the-art accuracy on CIFAR-10, CIFAR-100, SVHN, and ImageNet (without additional data). On ImageNet, we attain a Top-1 accuracy of 83.5% which is 0.4% better than the previous record of 83.1%. On CIFAR-10, we achieve an error rate of 1.5%, which is 0.6% better than the previous state-of-the-art. Augmentation policies we find are transferable between datasets. The policy learned on ImageNet transfers well to achieve significant improvements on other datasets, such as Oxford Flowers, Caltech-101, Oxford-IT Pets, FGVC Aircraft, and Stanford Cars.

1. Introduction

Deep neural nets are powerful machine learning systems that tend to work well when trained on massive amounts of data. Data augmentation is an effective technique to increase both the amount and diversity of data by randomly “augmenting” it [3, 54, 29]; in the image domain, common augmentations include translating the image by a few pixels, or flipping the image horizontally. Intuitively, data augmentation is used to teach a model about invariances in the data domain: classifying an object is often insensitive to

horizontal flips or translation. Network architectures can also be used to hardcode invariances: convolutional networks bake in translation invariance [16, 32, 25, 29]. However, using data augmentation to incorporate potential invariances can be easier than hardcoding invariances into the model architecture directly.

Dataset	GPU hours	Best published results	Our results
CIFAR-10	5000	2.1	1.5
CIFAR-100	0	12.2	10.7
SVHN	1000	1.3	1.0
Stanford Cars	0	5.9	5.2
ImageNet	15000	3.9	3.5

Table 1. Error rates (%) from this paper compared to the best results so far on five datasets (Top-5 for ImageNet, Top-1 for the others). Previous best result on Stanford Cars fine-tuned weights originally trained on a larger dataset [66], whereas we use a randomly initialized network. Previous best results on other datasets only include models that were not trained on additional data, for a single evaluation (without ensembling). See Tables 2.3, and 4 for more detailed comparison. GPU hours are estimated for an NVIDIA Tesla P100.

Yet a large focus of the machine learning and computer vision community has been to engineer better network architectures (e.g., [55, 59, 20, 58, 64, 19, 72, 23, 48]). Less attention has been paid to finding better data augmentation methods that incorporate more invariances. For instance, on ImageNet, the data augmentation approach by [29], introduced in 2012, remains the standard with small changes. Even when augmentation improvements have been found for a particular dataset, they often do not transfer to other datasets as effectively. For example, horizontal flipping of images during training is an effective data augmentation method on CIFAR-10, but not on MNIST, due to the different symmetries present in these datasets. The need for automatically learned data-augmentation has been raised recently as an important unsolved problem [57].

In this paper, we aim to automate the process of finding an effective data augmentation policy for a target dataset. In our implementation (Section 3), each policy expresses several choices and orders of possible augmentation opera-

*Equal contribution.

Dataset	Model	Baseline	Cutout [12]	AutoAugment
CIFAR-10	Wide-ResNet-28-10 [67]	3.9	3.1	2.6±0.1
	Shake-Shake (26 2x32d) [17]	3.6	3.0	2.5±0.1
	Shake-Shake (26 2x96d) [17]	2.9	2.6	2.0±0.1
	Shake-Shake (26 2x112d) [17]	2.8	2.6	1.9±0.1
	AmoebaNet-B (6,128) [48]	3.0	2.1	1.8±0.1
	PyramidNet+ShakeDrop [65]	2.7	2.3	1.5 ± 0.1
Reduced CIFAR-10	Wide-ResNet-28-10 [67]	18.8	16.5	14.1±0.3
	Shake-Shake (26 2x96d) [17]	17.1	13.4	10.0 ± 0.2
CIFAR-100	Wide-ResNet-28-10 [67]	18.8	18.4	17.1±0.3
	Shake-Shake (26 2x96d) [17]	17.1	16.0	14.3±0.2
	PyramidNet+ShakeDrop [65]	14.0	12.2	10.7 ± 0.2
SVHN	Wide-ResNet-28-10 [67]	1.5	1.3	1.1
	Shake-Shake (26 2x96d) [17]	1.4	1.2	1.0
Reduced SVHN	Wide-ResNet-28-10 [67]	13.2	32.5	8.2
	Shake-Shake (26 2x96d) [17]	12.3	24.2	5.9

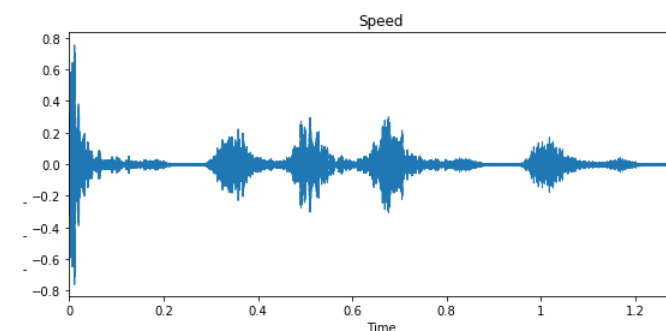
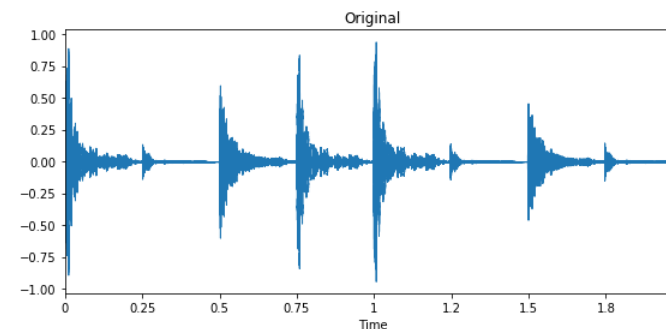
Test error-rate on some common benchmark.



Data augmentation also happens in other areas.

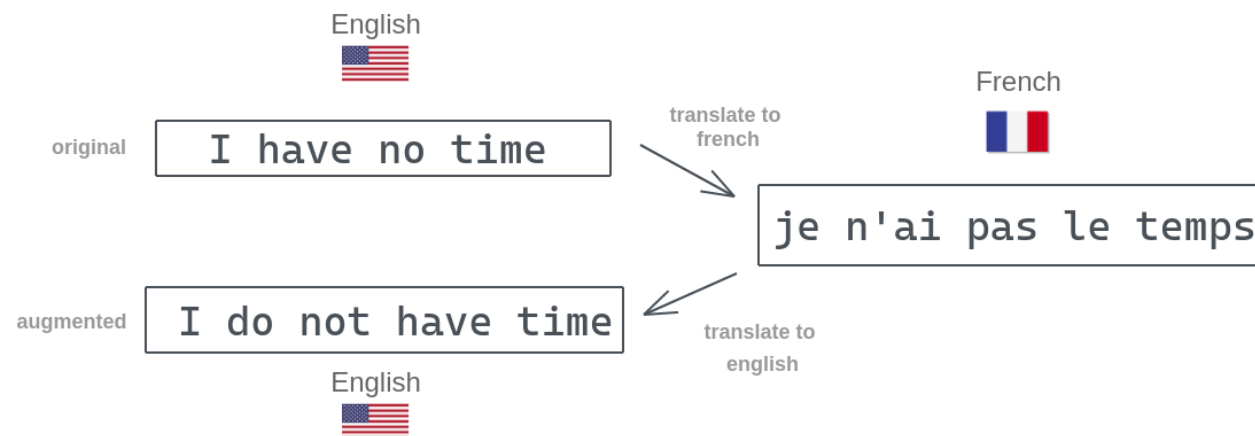
Audio:

- Speeding up / slowing down signal
- Modulating pitch
- Adding background noise
- Cropping / shifting audio



Natural Language Processing:

- Tricky to change text without changing meaning.
- Synonym replacing (weak)
- Backtranslation one option →



Today's Learning Objectives



Be able to answer:

- What optimizers are used for neural networks?
 - ~~What is stochastic gradient descent?~~
 - ~~What is momentum?~~
 - ~~What are weight decay and early stopping?~~
 - ~~What are learning rate schedules?~~
 - ~~What are adaptive learning rates?~~
- ~~How is data processed for neural networks?~~
 - ~~Why should data be normalized?~~
 - ~~What is data augmentation and why do it?~~
- What are best practices for network initialization?



Gradient descent is sensitive to initialization - where do we initialize?

- We are optimizing weights, so the question is what are the initial weights?

Can you give me some bad ideas?

One Intuition: We would like neurons to represent different things so at least some of them find good things during the optimization.

Another Intuition: Inputs being normalized seemed important ...
...and the output of each of our layers is the input to the next one!



Diversity is easy in high dimensions:

Suppose $w_1, w_2 \in \mathbb{R}^d$ are random unit norm vectors, they are increasingly likely to be orthogonal as d increases (similar idea to the last question in assignment 1)

$$P\left((w_1, w_2) \geq \sqrt{\frac{\log(d)}{d}}\right) < \frac{1}{d}$$

Initialize weight vectors as small random numbers.

What distribution? Depends on your activation function.

Bias vectors typically just set to zero.



Xavier initialization derived from sigmoid/tanh to try keeping variance constant across layers (on average):

$$w_i \sim \mathcal{N}\left(0, \frac{1}{d_{in}}\right)$$

Some alternative version use slightly different distributions. All meant to stabilize sigmoid/tanh.

$$w_i \sim \mathcal{N}\left(0, \frac{2}{d_{in} + d_{out}}\right)$$

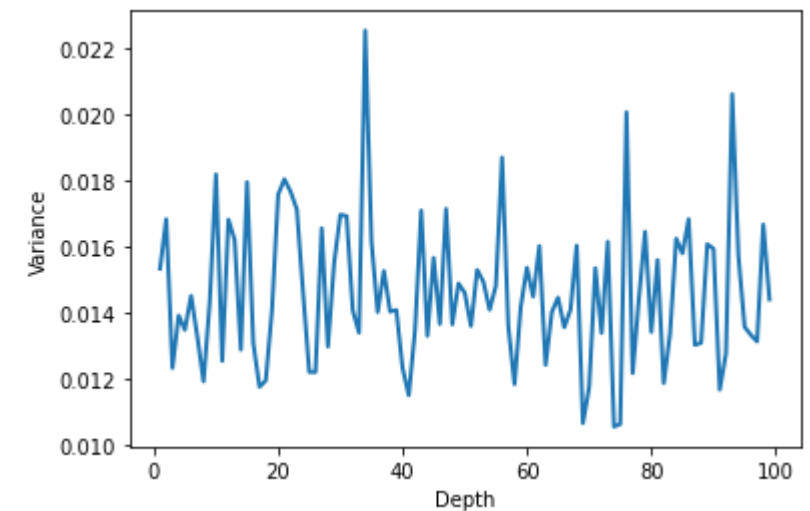
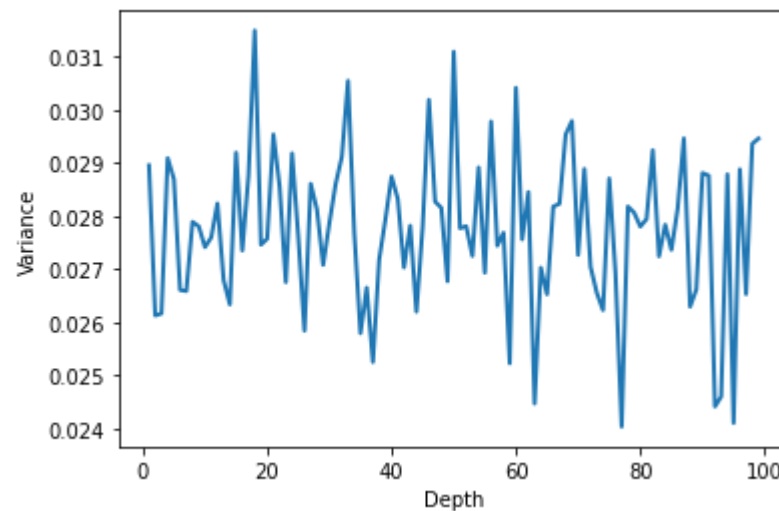
$$w_i \sim U\left(-\sqrt{\frac{2}{d_{in} + d_{out}}}, \sqrt{\frac{2}{d_{in} + d_{out}}}\right)$$



Xavier initialization derived from sigmoid/tanh to try keeping variance constant across layers (on average):

$$w_i \sim \mathcal{N}\left(0, \frac{1}{d_{in}}\right)$$

```
def siglayer(x):  
    std = np.sqrt(1/(x.shape[0]))  
    w = np.random.randn(x.shape[0], x.shape[0])*std  
    b = np.zeros((x.shape[0],1))  
    return 1/(1+np.exp(-np.dot(w,x)-b))  
  
x = np.random.randn(512,50)  
v = []  
for l in range(100):  
    x = siglayer(x)  
    v.append(np.var(x))
```



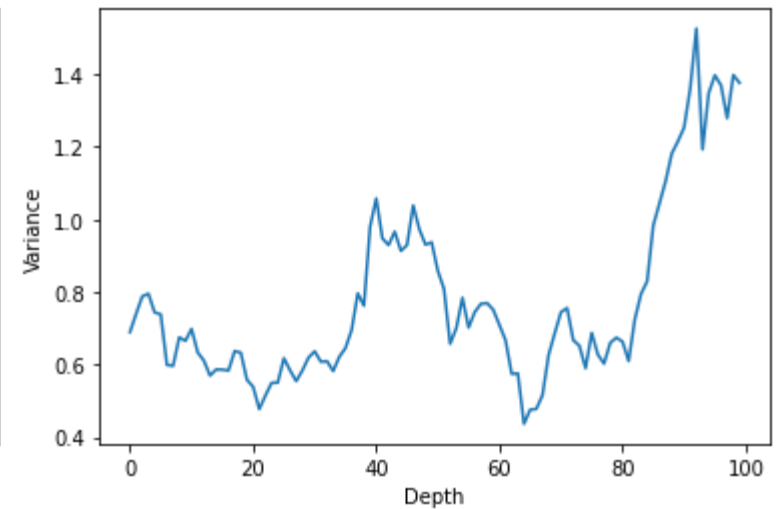
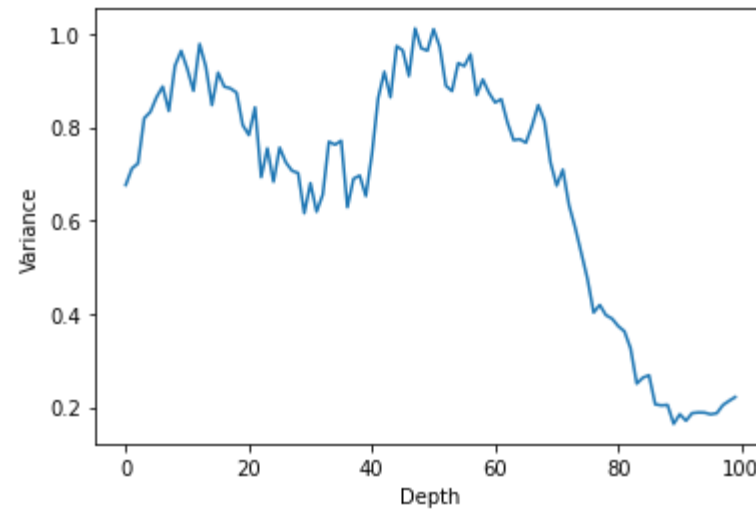
Understanding the difficulty of training deep feedforward neural networks - Glorot, X. & Bengio, Y. (2010)



He initialization derived from ReLU/LeakyReLU to try keeping variance constant across layers (on average):

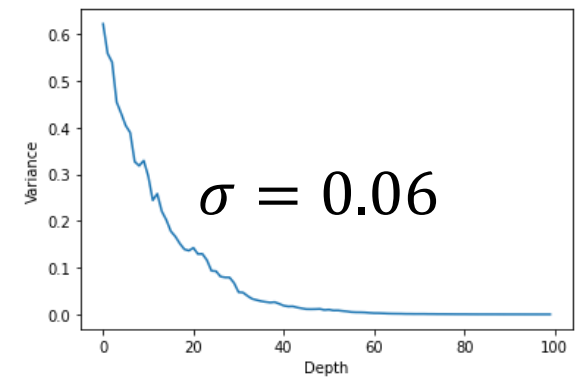
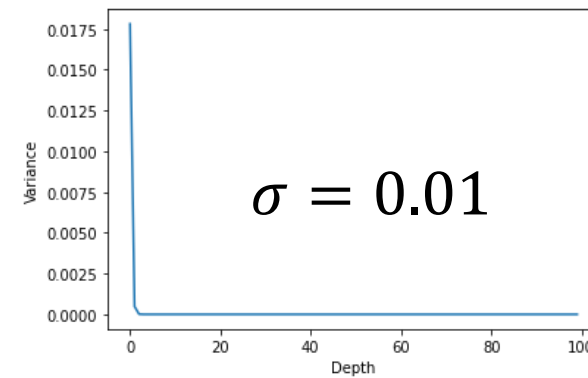
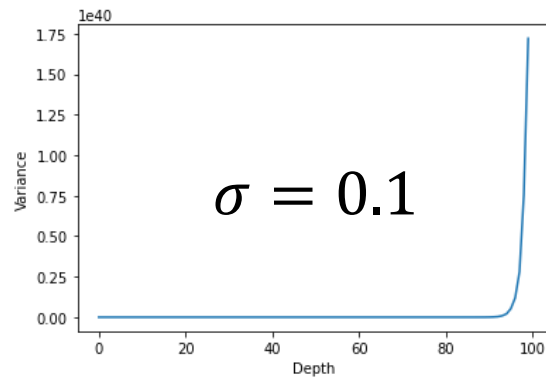
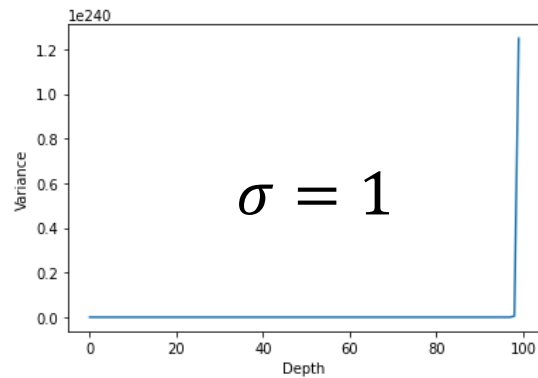
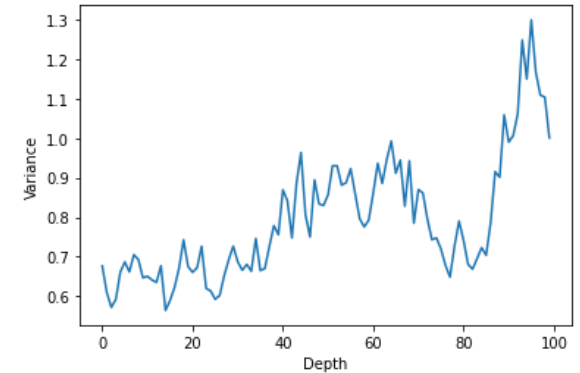
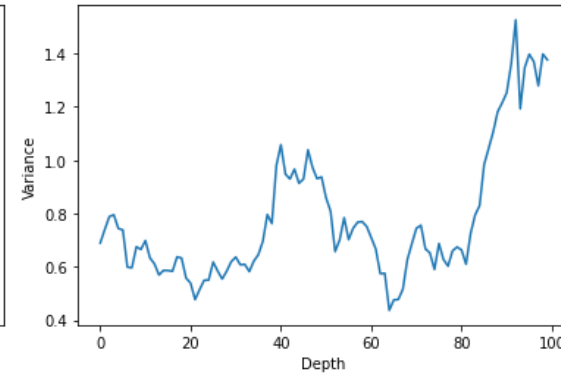
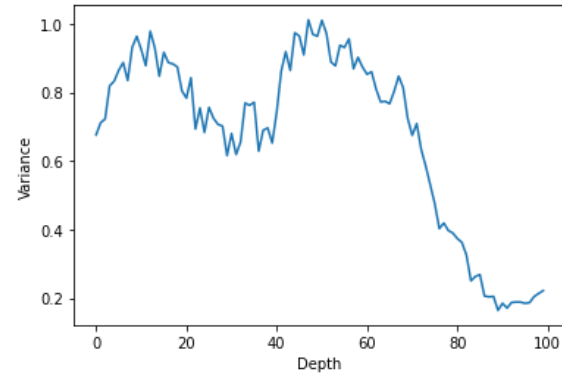
$$w_i \sim \mathcal{N}\left(0, \frac{2}{d_{in}}\right)$$

```
def relu_layer(x):  
    std = np.sqrt(2/(x.shape[0]))  
    w = np.random.randn(x.shape[0], x.shape[0])*std  
    b = np.zeros((x.shape[0],1))  
    return np.maximum(0,np.dot(w,x)+b)  
  
x = np.random.randn(512,50)  
v = []  
for l in range(100):  
    x = relu_layer(x)  
    v.append(np.var(x))
```





$$\sigma = \sqrt{\frac{2}{512}} = 0.0625$$



Initialization matters a lot at large depths. Variance may explode or drop to zero.

Today's Learning Objectives

Be able to answer:

- What optimizers are used for neural networks?
 - What is stochastic gradient descent?
 - What is momentum?
 - What are weight decay and early stopping?
 - What are learning rate schedules?
- How is data processed for neural networks?
 - Why should data be normalized?
 - What is data augmentation and why do it?
- What are best practices for network initialization?

